GALILEO Group Second Deadline Report (31 october 2025)

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

November 16, 2025

Contents

1 Introduction

1.1 Context and Objective

The preceding project phase involved the **assessment of query execution** on the database for each dataset, followed by the processing of the results obtained from our implementation using the `galois_eval.py` script.

The primary objective of the current task is to detail the **development process** implemented to replicate the following **baselines**, specifically utilizing the **Internal Knowledge datasets**:

- **SQL Baseline**: The system directly prompts the *Large Language Model* (LLM) with an **SQL query** as the input.
- **NL (Natural Language) Baseline**: The system employs a **Natural Language prompt** as the direct input to the LLM,
- **Palimpzest Baseline**: This baseline is designed to evaluate the pure **reasoning and synthesis capabilities** of the LLM when operating within a **Retrieval-Augmented Generation (RAG)** paradigm.

Note on Implementation: The current setup represents a foundational RAG configuration and necessitates further methodological refinement to fully conform to the advanced data handling and inference strategies detailed in the original Galois paper.

1.2 Report Structure

For each implemented baseline, **synoptic tables** reporting the derived evaluation measures and metrics will be presented. This introductory section establishes the methodological context; the subsequent sections will proceed with a detailed analysis of each individual baseline.

2 Core Architectural Design and Execution Flow

2.1 Architectural Design Patterns for Modularity

The codebase's central logic is structured around established software design patterns to ensure high modularity, extensibility, and maintainability across heterogeneous components.

Adapter Pattern (`llm_wrappers.py`): This pattern standardizes the interface for interacting with heterogeneous LLM providers (e.g., Gemini, Watsonx). By abstracting the provider-specific connection details, it enables the core execution engine to interact with all models through a uniform API.

Factory Pattern (`llm_factory.py`): Implemented as a simple factory method, this pattern decouples client components (e.g., `sql_baseline.py`, `nl_baseline.py`) from the specific instantiation logic of concrete LLM wrapper objects. Extensibility to new providers is achieved solely by updating the internal `PROVIDER_MAP` registry.

Template Method Pattern (`LLMBaseWrapper` in `llm_wrappers.py`): The abstract `LLMBaseWrapper` defines the invariant sequence (the *template*) for the life cycle of the LLM instance retrieval. It establishes the mandatory sequence while deferring the implementation of the provider-specific connection details (via `create_llm_instance()`) to concrete subclasses.

Chain/Pipeline Pattern (LCEL) (`build_lcel_chain.py`): This pattern, realized using LangChain Expression Language (LCEL), defines a structured, sequential execution flow:

Context Generation → Prompt Construction → LLM Invocation

This structured approach ensures predictable and auditable data transformation across the execution path.

2.2 Standardized Execution Workflow (The Pipeline)

The execution workflow is structured into a four-stage pipeline, ensuring robust and consistent processing across all baseline types:

1. **Context Generation**: The `get_db_context` function dynamically prepares the necessary input context. This includes retrieving the database schema for SQL/NL baselines or simulating *Retrieval-Augmented Generation* data for the Palimpzest baseline, based on the requested execution type.

2. **Prompt Engineering and Injection**: The derived context is programmatically injected into a sophisticated `ChatPromptTemplate`. This stage manages advanced prompting logic, including the insertion of mandatory system instructions and the enforcement of the required JSON output structure via a `PydanticOutputParser`.

3. **LLM Invocation**: The fully prepared and structured prompt is transmitted to the configured LLM instance, which was instantiated via the Factory/Adapter layers.

4. **Response Standardization and Metadata Enrichment**: The `parse_llm_response` function enforces cross-provider output standardization. This involves the mandatory validation and parsing of the requisite JSON structure returned by the LLM. It then enriches this output into the `FULL_JSON` format by extracting the `token consumption` from the LLM's response metadata and internally calculating and appending the `LLM response latency`.

This standardized and enriched data is essential for downstream performance evaluation.

2.3 Data Context and Output Standardization

This section defines the two primary contexts used to condition the Large Language Models (LLMs) and establishes the **standardized JSON output format** mandatory for all baseline evaluations (NL, SQL, and Palimpzest).

2.3.1 Data Context Categorization

The evaluation utilizes two distinct dataset contexts, governing the type of factual information provided to the LLM during prompting:

- **Internal Knowledge (IK):** In these datasets, the model relies exclusively on its pretrained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and the database schema for both the **SQL** and **NL** baselines, but no factual data retrieved from the database.

- **Model Context (MC):** These datasets are designed to enrich the prompt with external information retrieved from a knowledge base. For this project task, MC datasets were **not used** in either the NL or SQL baselines, but they form the basis of the **Palimpzest** baseline to evaluate its Retrieval-Augmented Generation (RAG) capabilities.

2.3.2 Standardized Output Format

Every response produced by any baseline is written in a JSON file that fits the mandatory **FULL JSON** format. This rigorous structure ensures consistent computation of metrics across all providers and baselines.

```
{
    "result_set": [
        { "originaltitle": "The Three Musketeers"},
        { "originaltitle": "The Count of Monte Cristo"}
    ],
    "time": 1.84,
    "tokens": 312
}
```

Figure 1: FULL JSON Output Format Example

Field Name	Description
result_set	Denotes the structured LLM output. This must be an array of JSON objects corresponding to the query's selected columns.
time	Represents the total time taken by the LLM to generate the response.
tokens	Indicates the number of tokens processed during the interaction.

3 Baseline NL

The Baseline NL module provides a reference framework for evaluating Large Language Models (LLMs) through direct interaction via natural language prompts. This approach assesses the model's intrinsic reasoning and knowledge by prompting it with natural language (NL) questions, optionally enriched with structural and contextual information derived from a database schema or a specific query results. All outputs are standardized in a unified FULL JSON format to facilitate downstream evaluation.

3.1 General Procedure

1. **Prompt Loading:** Natural language prompts are loaded from dataset-specific resources (**queries_<dataset-name>.txt**). Each prompt corresponds to a query from the same dataset.

2. **LLM interaction:** Prompts are processed through a **LangChain-based pipeline**, which integrates:

   - The general task definition and LLM role specification.
   - The corresponding database schema, extracted via DuckDB.
   - Optional contextual data (e.g. query results for Model Context datasets).
   - The natural language question itself

3. **Response Parsing and Storage:** The model's output is parsed using a **PyddanticOutputParser**, ensuring compliance with the required FULL JSON format. Each response is stored in an individual JSON file, forming a consistent source for evaluation.

3.2 Architecture:

The architecture is composed of three main components:

1. **Configuration Layer:**

   - API keys for the supported providers (**IBMWatsonx** and **Google Gemini**) are loaded form a **.env** file.
   - The **Config_Loader** selects the target LLM provider, and the appropriate wrapper is instantiated through **get_llm_wrapper()**.
   - Each wrapper extends the **LLMBaseWrapper** class, implementing provider-specific logic for model initialization, token counting, and response handling.

2. **Chain Construction:**

   The **build_lcel_chain()** method constructs a **LangChain Expression Language (LCEL)** pipeline comprising:

   - **Database Context Retrieval: get_db_context()** extracts schema information and, for Model Context datasets, executes the original SQL query using DuckDB database instance of corresponding dataset.

- **Prompt Composition: ChatPromptTemplate** merges two layers, a **SYSTEM_PROMPT** defining the LLM's role and behavior, and a **HUMAN_PROMPT** containing the NL question and formatting directives.
- **Structured Output Parsing:** The **PydanticOutputParser** enforces adherence to the predefined FULL JSON schema.

3. **Execution Flow**

- For each dataset, SQL queries and corresponding NL prompts are loaded.
- The **(prompt, query)** pairs are processed through **chain.invoke()**.
- Responses are parsed, validated, and augmented with the metadata such as execution time and token usage.
- The resulting files are serialized via **save_baseline_to_json()** for evaluation purposes.

4 Baseline SQL

The **SQL Baseline** module is engineered to extend the core evaluation framework to scenarios where the Large Language Model (LLM) is interrogated via structured **SQL statements** rather than Natural Language (NL) queries.

The primary objective is to quantitatively assess the capacity of different LLM providers to interpret complex SQL syntax, perform logical reasoning over the supplied database schema and data, and output a rigorously structured JSON response compatible with the established evaluation metrics used for the NL baseline.

—

4.1 Standardized Execution Pipeline

The Baseline SQL leverages a five-stage execution pipeline, conceptually aligning with the NL baseline but specifically tailored for structured input processing:

1. **SQL Statement Retrieval**: All target SQL statements for a given dataset are systematically loaded from the corresponding `queries_<dataset>.sql` file via the dedicated `load_queries_from_folder()` utility function.
2. **Schema and Context Augmentation**: The designated DuckDB database instance is initiated, and its schema definition is extracted. Datasets are segregated into two distinct contexts for model conditioning:
   - *Internal Knowledge(IK)*: The database schema is provided to the LLM as external context.
   - *Model Contexts(MC)*: This dataset isn't used in accordance with the Galois Paper.
3. **Prompt Composition and Contextualization**: The full prompt artifact is constructed by retrieving and combining pre-defined templates from `constants.py`. Specifically, the process involves two steps:
   - The appropriate `System Prompt` (either for IK or the currently unused MC datasets) is selected to define the high-level task directive (Context data and JSON formatting).
   - The `Human Prompt` template, specific to the SQL Baseline, is then populated with the core SQL statement itself.
4. **LLM Invocation and Output Validation**: The prepared prompt is dispatched to the selected LLM provider (Gemini or Watsonx). The resultant answer is subjected to validation via a `PydanticOutputParser`, enforcing a mandatory `FULL JSON` schema which includes the fields `result_set`, `time`, and `tokens`.
   Rigorous error handling ensures that parsing failures result in the logging of the raw content and the return of an empty `result_set` with timing metadata.
5. **Result Serialization**: The output for each query is serialized into a JSON file and stored in a provider-specific subdirectory. This standardized output format facilitates direct consumption by the evaluation scripts, ensuring consistent computation of metrics across all baselines.

4.2 Architectural Implementation and Provider Decoupling

The implementation adheres strictly to the **Adapter** and **Factory** patterns, ensuring robust decoupling from specific LLM providers. Instead of segmenting core pipeline execution logic, the architecture centralizes control, allowing a single workflow to dynamically switch between providers.

Centralized Execution Flow (`execute_baseline_sql`): The primary execution control is managed by the single function `execute_baseline_sql`. This function orchestrates the entire process, encompassing data selection (via CLI or configuration), context retrieval, and result serialization. Crucially, `execute_baseline_sql` does **not** contain provider-specific code; it delegates responsibilities to specialized helper utilities:

- `build_lcel_chain`: This function is responsible for instantiating the correct LLM adapter based on the provider specified in the YAML configuration file. This maintains the Factory pattern's principle of decoupling the creation logic.
- `parse_llm_response`: Standardizes and enriches the output, as detailed in the pipeline, ensuring a uniform `FULL_JSON` structure regardless of the underlying LLM.
- `save_response_to_JSON`: Handles the final result persistence and serialization for all providers uniformly.

Provider Abstraction (Zero-Code Provider Components): Consequently, the implementation avoids the need for dedicated provider-specific execution modules. Instead, the abstraction is achieved entirely within the LLM Adapter layer. This central layer manages the necessary provider-specific API calls and setup details, which are configured dynamically based on the model selected in the YAML configuration. This approach facilitates a "zero-code execution logic" at the component level outside of the Adapter, ensuring that all providers utilize the same standardized workflow and that results are inherently comparable using the identical `FULL_JSON` output schema and evaluation criteria.

5 Results and Evaluation

The final quantitative outcomes for the SQL Baseline are integrated with the NL results, summarized in tables detailing the **F1-CELL**, **CARDINALITY**, and **TUPLE-CONSTRAINT** metrics, along with their average scores, average inference time (**latency**), and total token consumption.

The results obtained from the Baseline NL and SQL baseline modules were evaluated using standard metrics to assess the performance of the LLMs in both Natural Language (NL) and SQL contexts. The model used for the **gemini** provider is: **gemini-2.5-flash**, instead for **IBMWatsonx** provider we used **meta-llama/llama-3-3-70b-instruct**. All systems are tested on the datasets described in Table 2 of the previous project task, excluding Premier and Fortune, as the Galois authors did.

Metric	NL	SQL
F1-CELL	0.391	0.257
CARDINALITY	0.644	0.571
TUPLE-CONSTR.	0.182	0.119
#TOKENS(M)	39.965	0.063
AVG-TIME (s)	4.812	4.858
AVG-SCORE	**0.406**	**0.315**

Table 1: GEMINI results

Metric	NL	SQL
F1-CELL	0.436	0.523
CARDINALITY	0.701	0.718
TUPLE-CONSTR.	0.212	0.243
#TOKENS(M)	0.015	0.012
AVG-TIME (s)	5.353	4.485
AVG-SCORE	**0.450**	**0.495**

Table 2: WATSONX results

Some comments about the results in the tables above:

The analysis of results across different runs reveals that quality and cost metrics are intrinsically linked to the precision of the underlying Large Language Model (LLM) (various models have different reasoning ability and prompt sensibility as you can see in the result metrics reported above) this is one of the reasons for which LLM's performance can fluctuate, impacting key metrics. Another trivial factor is the LLM's tendency to generate values or parameter names that are semantically similar but syntactically different from the expected attribute names or those reported in the ground truth JSON files, this discrepancy, even if minor, can lead to false negatives.

Finally, variability is reflected in costs: the number of tokens consumed (#TOKENS(M)) and the average execution time (AVG-TIME (s)) can vary significantly across providers, depending on different internal prompting strategies and context limitations.

6 Baseline Palimpzest

The baseline aims to automatically retrieve relevant text segments (from Markdown or plain text files) to construct additional contextual information for a Retrieval-Augmented Generation (RAG) pipeline. The file resources that we used for this baseline was retrieved from the GALOIS repository within the core/src/test/resources/rag-fortune and core/src/test/resources/rag-premier folders. In short, it builds a semantic knowledge base from textual documents and indexes them using embeddings and FAISS.

Talking about FAISS, it is an open-source library developed by Facebook AI Research that provides efficient similarity search and clustering of dense vectors.

Th reason why we use this library is that we need something that allows us to perform fast vector indexing and similarity searches over large datasets of text embeddings, in a reasonable time and CPU usage.

For what concerns the resources usage (CPU, GPU) we used only the CPU for ensuring portability and easy baseline testing without GPU support. Actually the most of the times, the baseline was tested on a machine with a very low GPU power (a Laptop with a integrated GPU, not a dedicated one); furthermore when we try to run the baseline using also the GPU the running time needed to download the dedicated Python Library for GPU support was much higher than the time needed to download the CPU ones, so finally we decided only to use the latter. For the same time saving reasons we used a light sentence transformer model (**sentence-transformers/all-MiniLM-L6-v2**) for the sentence embeddings generation, in fact it has a good trade-off between the running time speed and the semantic precision and also avoids the overhead of large transformer models while still maintaining strong semantics representations.
Since our implementation needs to be easily deployable and testable on different machines, we decided to avoid heavy models that require high computational resources.
However as you can infer, there are some differences between our architecture and the full Palimpzest system presented in the reference Galois paper: The core difference lies in the execution methodology: **Single-Step RAG** vs. **Multi-Step Procedural Optimization**. Summarizing our baseline utilizes a straightforward, single-step RAG pipeline built using a LangChain Expression Language (LCEL) chain. This design focuses on cost efficiency and establishing a floor on quality for context-augmented retrieval, in the other hand the Palimpzest system in the GALOIS paper is a sophisticated, highly-optimized AI workload framework based on an ETL-like (Extract, Transform, Load) procedural architecture.

Our approach should therefore be interpreted as the most cost-efficient, baseline implementation of RAG augmentation, explicitly contrasting with the cost-intensive, quality-maximizing procedural methodology of the full Palimpzest framework.

6.1 Overall Architecture

Generally the baseline consists of three main components:

- **MarkdownRAGBackend class**: It handles file loading, chunking, embedding and FAISS indexing.
- **pz_context()**: main method that uses the backend to build contextual input (relevant chunks) for the Palimpzest model.
- **Backend**: Represents all the "behind the scenes" logic.

6.2 Initialization (__init__)

Here a HuggingFaceEmbeddings instance is created to generate text embeddings using a specified model, the chosen model is **sentence-transformers/all-MiniLM-L6-v2** as already seen.

6.3 Loading and Indexing (Load_and_index)

This method performs the full data preparation and indexing pipeline:

- Searching for a .md, .txt or extensionless files
- Converts each file into LangChain documents, preserving metadata.
- The text is divided in chunks (128 tokens per chunk for premier and 400 tokens per chunk for fortune) balancing semantic coherence and retrieval precision, and also preserves contextual continuity across chunks setting an overlap of 20 tokens.
- Creation of FAISS index from the generated embeddings for efficient similarity search.

6.4 Retrieval of Relevant Chunks (retrieve)

Essentially the method returns the **top_k** most semantically similar text chunks for a given query, so it performs the semantic retrieval.

6.5 Saving and Loading Index

Finally the FAISS index is saved in a persistent way to avoid re-indexing on subsequent runs, and it can be loaded back when needed.

We would like to emphasize that the reason we used the text files available in the GALOIS repository as resources, rather than, for instance, a subset of records from the Fortune or Premier datasets (or similar sources), is that the original idea behind the Palimpzest baseline (as stated in the GALOIS paper) is to evaluate the LLM's capabilities in a RAG paradigm using external textual resources that are not part of the datasets themselves.

6.6 Results and Evaluation

Metric	NL	SQL
AVG-SCORE	0.328	0.347
#TOKENS(M)	0.003	0.003

Table 3: PALIMPZEST results

Some considerations: The baseline shows modest but consistent quality scores for both Natural Language (NL), SQL inputs, showing a consistent performance for his RAG purpose. Trivially performs marginally better when the query is supplied as a formalized SQL statement versus a less structured NL question. This aligns with the general observation

that declarative SQL inputs reduce ambiguity for the LLM, even in an in-context RAG setting. In summary, the RAG-based Palimpzest implementation delivers a minimal-cost solution, but its current quality performance suggests there is significant scope for improvement, particularly by enhancing the prompt construction, chunk selection, or the iterative refinement steps inherent to many RAG/Palimpzest systems.

6.7 Unified Command-Line Interface (CLI) Flow

The central execution control is consolidated within `src/main.py`, providing a unified interface for the orchestration of both NL, SQL and Palimpzest baselines. The CLI exposes the following essential control arguments:

Argument	Type	Function
`datasets`	Positional	Accepts zero to multiple dataset identifiers (e.g., `WORLD`, `GEO`).
`-mode`	Option	Selects the execution type: `nl`, `sql`, `both`, `pzsql (Palimpzest with sql)` or `pznl (Palimpzest with nl)`.
`-provider`	Option	Overrides the default LLM provider defined in the configuration (`gemini` or `watsonx`).

Table 4: Essential CLI Control Arguments for Baseline Execution

High-Level Logical Flow The overall execution sequence is dictated by the input received from the CLI:

1. **Initialization**: Argument parsing is performed to determine the list of target datasets, prioritizing the CLI input over the default configuration settings.
2. **Conditional Invocation**: Based on the selected `-mode`, the system conditionally invokes the NL, SQL, PZSQL, PZNL or both SQL & NL baselines.
3. **Task Dispatch and Persistence**: The task is dispatched to the provider-agnostic implementation, and the resulting `FULL_JSON` output files are persistently stored for evaluation.

7 Future Work

For next tasks, we will consider improving and extending the actual logical and structural retrieval information process: In particular will focus on the integration and deployment of the GALOIS framework to significantly improve the accuracy and efficiency of structured data retrieval. This integration is crucial for addressing the limitations inherent in directly prompting Large Language Models (LLMs) with complex Natural Language (NL) or SQL queries, which often result in low precision and recall.

Our implementation effort will concentrate on two main areas: **Logical Plan Optimization** and **Physical Operator Design**.

- **Logical Plan Optimization**: We will implement the **Filter-LLMScan** operator, which allows selection conditions to be pushed down directly to the LLM. This is a critical step because the choice of which conditions to push is complex; pushing down all conditions can reduce token cost but often degrades result quality due to the LLM's difficulty in processing complex requests.
- **Physical Operator Design**: We will implement and evaluate both alternative physical scan operators supported by GALOIS:
  - **Table-Scan**: A single prompt requests the LLM to return the entire result set in a structured JSON format. This approach is token-efficient but may produce lower quality results.
  - **Key-Scan**: This two-step operator first retrieves the key values for the relevant tuples and then iteratively queries the LLM to populate the remaining attribute values for each key.

Summarizing the meaning: to decide between these two strategies described above, we will incorporate GALOIS's confidence threshold (τ) mechanism16. By computing the confidence of the LLM in retrieving key values, we can select Key-Scan when confidence is high (improving quality) or revert to Table-Scan when confidence is low (for more reliable, albeit potentially less complete, data retrieval).

Furthermore for what concerns the Palimpzest baseline, the system metrics reported in the Palimpzest section lacks a comparison with SQL or NL metrics, since in the NL and SQL baselines we have take into account only the IK datasets and not the MC ones, and this will be a future task to be done.

8 Contributions

Alessio Demo (Badge number: 2142885)

- **First Deadline:** Developed function database connection and instance creation of DuckDB database manager for each dataset using the ingest files (.duckdb files). Developed function for retrieving the SQL queries from .sql files, processing the queries and executing them on DuckDB database instance and finally store it in a json file.
- **Second Deadline:** Developed the Baseline NL function for interacting with the LLM. "Early-stage" Palimpzest baseline implementation.

Francesco Pivotto (Badge number: 2158296)

- **First Deadline:** Implemented a modular project structure by relocating core logic to `src/` and consolidating configuration and database management logic within dedicated utility modules (`config/loaders.py`). Established the framework for multi-provider support by adding initial integration for the `Grok LLM`, including dedicated configuration points (`.env`, `config.yaml`, `loaders.py`).
- **Second Deadline:** Completed initial implementation of the SQL baseline, including the core logic for connecting to `DuckDB` and structuring query execution via the Watsonx provider. Fixed critical issues in environment variable loading, ensuring robust and standardized retrieval of secrets and settings via the central configuration object (using `loaders.py`). Modularized the `sql_baseline` module, incorporating the LCEL Pattern to prepare for dynamic LLM selection. Centralized general utility functions by moving helpers into `baseline_tools`. Introduced the dedicated `llm_wrapper` class to handle all LLM connection logic, API parameter management, and calls.

Giorgia Amato (Badge number: ...)